SINGAPORE UNIVERSITY OF SOCIAL SCIENCES

ICT133

Proctored Online Exam - July Semester 2025

Structured Programming

Thursday, 13 November 2025 01:00 pm - 03:00 pm

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This examination contains **9** questions.
2. Answer all parts of each question in the corresponding answer box.
3. All answers must be typed in the answer box and submitted in Examena.
4. This is a/an **Closed-book** examination.
5. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it.
6. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.
7. **One** A4-sized (double-sided) hardcopy Cheat Sheet allowed.

At the end of the examination.
To end the examination, please click on the "Submit and end exam" button to submit your answers.

THE UNIVERSITY RESERVES THE RIGHT NOT TO MARK YOUR SCRIPT IF YOU FAIL TO FOLLOW THESE INSTRUCTIONS.

ICT133 Copyright © 2025 Singapore University of Social Sciences (SUSS)

(Full marks: 100)

Question 1

Express how the following sequence of statements behaves. What is the output when the function testSelection() is called?

```
def testSelection():
    a = 6
    b = 5
    c = 4

    if a > b:
        print("one")
    elif a < c:
        print("two")

    if c < b:
        print("three")
```

A. "one"
B. "three":
C. "one" and "three"
D. No output

(5 marks)

Question 2

Express how the following sequence of statements behaves. What will be the output when the given code is executed?

```
for n in range(3,6):
    for i in range(2,n):
        if (n+i)%2 == 1:
            print(n+i)
            break
```

A. 5 7 7
B. 5 7 8
C. 5 7 9
D. 5 7 7 9

(5 marks)

Question 3

Express how the following sequence of statements behaves. What will be the output of the following Python program?

```
x = 0
while x < 6:
    x += 1
    if x == 3:
        continue
    if x == 5:
        break
    print(x)

```

A. 1 2
B. 1 2 3
C. 1 2 4
D. 1 2 5

(5 marks)

Question 4

What will be the output of line 10 and line 13 when the following Python code is applied?

```
1  def add1(a: list, b: list):
2      a = a + b
3      return a
4
5  def add2(a: list, b: list):
6      a.append(b)
7      return a
8
9  my_list = [1,2,3]
10 print( add1(my_list, [4]) )
11
12 add2(my_list, [5])
13 print( my_list )
```

A. [1, 2, 3, 4] and [1, 2, 3, 4, [5]]
B. [1, 2, 3, [4]] and [1, 2, 3, 4, 5]
C. [1, 2, 3, 4] and [1, 2, 3, [5]]
D. [1, 2, 3, [4]] and [1, 2, 3, [5]]

(5 marks)

Question 5

What output will be produced when the following Python code is applied?

```
my_dict = {'a': 1, 'b': 2, 'c': 3}
my_dict['d'] = 4
print(my_dict.keys())
```

A. [1, 2, 3, 4]
B. dict_keys(['a', 'b', 'c', 'd'])
C. ['a': 1, 'b': 2, 'c': 3, 'd': 4]
D. ('d', 'a', 'b', 'c')

(5 marks)

Question 6

Read the entire question before attempting. You need to submit each part individually.

Question 6a

Write a function *insertSpaces(s: str) -> str* that performs the following:

- Insert a space between each character in the given string s, ensuring that characters already separated by spaces remain unchanged.
- The input s may contain multiple words, such as "basic python programming", and any existing spaces should remain intact.
- Leading or trailing spaces in s should not affect the result.
- The function should return the transformed string.
- Hint: Consider using *strip()* to handle leading and trailing spaces effectively.

Sample Runs:

```
print( insertSpaces("  apple  ") )          # Output: "a p p l e"
print( insertSpaces("hello") )               # Output: "h e l l o"
print( insertSpaces("hello world") )        # Output: "h e l l o w o r l d"
print( insertSpaces("ba nana") )            # Output: "b a n a n a"
print( insertSpaces("a b c") )               # Output: "a b c"
```

(15 marks)

Question 6b

Write a Python program that continuously takes user input and employs the following rules to process the entered string:

- Insert a space **between each character** in the given string `s`, ensuring that characters already separated by spaces remain unchanged.
- The program should repeatedly prompt for input until the user presses **Enter** without typing anything.
- Upon termination, print `"Program terminated."`.
- Remember to use the function created in Q6(a).

Sample Runs:

```
Enter a word (Press Enter to exit): apple
a p p l e

Enter a word (Press Enter to exit):     hello world
h e l l o w o r l d

Enter a word (Press Enter to exit): <ENTER>
Program terminated.
```

(5 marks)

Question 7

Designed by Samuel Morse and Alfred Vail in 1840s, the Morse code is an old system of encoding messages that is used to send telegraphic information using signals and rhythm. It uses short and long pulses - tones or lights - to represent letters and numbers.

Develop a Python application that solves the problem of encoding text messages into Morse code.

```
1 def main():
2     morseMapping = setupMorseMapping()
3     print(morseMapping)
4     textToMorse(morseMapping, "infile.txt", "outfile.txt")
5
6 main()
7
```

The function setupMorseMapping( ) is **predefined** and it generates a dictionary mapping each character to its morse code equivalent. The dictionary assigned to morseMapping at line 2 will look like:

{ 'A': '.-', 'B': '-...', 'C': '-.-.', 'D': '-..',
 'E': '.', 'F': '..-.', 'G': '--.', 'H': '....',
  ….. ….
  ….. ….
 'W': '.--', 'X': '-..-', 'Y': '-.--', 'Z': '--..', ' ': '/'}

The "infile.txt" at line 4 contains the text messages to be encoded, while "outfile.txt" will store the translated morse codes. Below is a sample of a "infile.txt" file:

SOS SOS
The exam is very difficult

The translated morse codes in "outfile.txt" will look like:

... --- ... / ... --- ...
- .... . / . -..- .- -- / .. ... / ...- . .-. -.-- / -.. .. ..-. .. .. -.-. ..- .-.. -

Write Python functions, such that the above Python application can execute. (You need to submit each part individually)

Question 7a

encode(morseMapping: dict, character: str) -> str:

- This function has TWO parameters: morseMapping, that is the dictionary populated from function *setupMorseMapping( )*, and character that the function must encode.
- For example, to encode "S", the function will look up the morseMapping dictionary and returns "..."
- If the given character cannot be encoded, the function returns "#"

(5 marks)

Question 7b

textToMorse(morseMapping: dict, infile: str, outfile:str):

- This function reads the text (multiple lines) from the infile, encodes each character into morse code and writes the translated result into the outfile.
- You may choose to insert space between the morse codes but is not a must-do requirement.
- Remember to make use of the function encode() created in Q7(a).

(20 marks)

Question 8

You are tasked with developing a Python game where the player starts with a seed score of 21 and rolls a six-sided dice to deduct from the seed score. The game follows these rules:

1. The player rolls a dice and subtracts the rolled value from the current seed score.
2. The goal is to reduce the seed score to exactly 0.
3. If the dice roll exceeds the current seed score, the seed score will not be deducted, and the player must roll again.

Note: no user input required.

Sample run:

```
Seed Score: 21
Roll: 3 ... Seed Score now: 18
Roll: 5 ... Seed Score now: 13
Roll: 6 ... Seed Score now: 7
Roll: 5 ... Seed Score now: 2
Roll: 4 ... Need to roll again
Roll: 2 ... Seed Score now: 0
Game Over!
```

(10 marks)

Question 9

Extend the Python game in Q8 where three players now compete to reduce their seed score of 21 to exactly 0 in each round. The game being developed will track player scores and determine the overall winner after three rounds.

Game Rules:

1. Players: The game consists of three players: "Tom", "Jim", and "Dan".
2. At the start, initialize a dictionary to store the player's name and default seed values.

score = { "Tom": [21, 21, 21], "Jim": [21, 21, 21], "Dan": [21, 21, 21] }

3. Rounds: The game consists of 3 rounds. Hence the list [21, 21, 21] represents the starting seed values for each round.
4. In each round:
    - Each player rolls a six-sided dice and subtracts the rolled value from their current seed score.
    - If the dice roll exceeds the current score, the roll is ignored, and the player must roll again.
5. Round Winner: The player who successfully reaches exactly 0 first in a round is declared the round winner.
6. Overall Winner: After three rounds, the player with the lowest total remaining seed score is the overall winner.

Note: no user input required.

Sample run:

```
Round 1:
Tom rolls: 3 ... Seed Score now: 18
Jim rolls: 5 ... Seed Score now: 16
Dan rolls: 6 ... Seed Score now: 15
====
Tom rolls: 6 ... Seed Score now: 12
Jim rolls: 6 ... Seed Score now: 10
Dan rolls: 4 ... Seed Score now: 11
====
Tom rolls: 6 ... Seed Score now: 6
Jim rolls: 2 ... Seed Score now: 8
Dan rolls: 1 ... Seed Score now: 10
====
Tom rolls: 4 ... Seed Score now: 2
Jim rolls: 1 ... Seed Score now: 7
Dan rolls: 5 ... Seed Score now: 5
====
Tom rolls: 2 ... Seed Score now: 0
Jim rolls: 5 ... Seed Score now: 2
Dan rolls: 6 ... Need to roll again
Tom reaches Seed Score: 0! Wins the round.

Updated Scores:
score = { "Tom": [0, 21, 21], "Jim": [2, 21, 21], "Dan": [5, 21, 21] }

Round 2 & 3 proceed similarly.

Final Scores:
score = { "Tom": [0, 1, 2], "Jim": [2, 0, 4], "Dan": [5, 0, 6] }
Overall Winner: **Tom** (lowest total remaining seed score)
```

(20 marks)

----- END OF PAPER -----